Klar – gjennomgang før publisering

davidtoska/elm-klar 1.0.0 · 80 eksponerte moduler, ca. 72 000 linjer Elm · gjennomgang 21. oktober 2026

> **Kort svar: nesten, men ikke ennå.**
> Kvaliteten på selve komponentene er uvanlig høy, og kompleksiteten er stort sett godt skjult. Men Elm krever at første publisering er **1.0.0**, og etter det kan offentlige typer ikke endres uten en ny hovedversjon. Det betyr at fem API-avgjørelser bør tas *før* du publiserer, ikke etter. Den viktigste er at `Remote` (loading/feil-tilstand) i dag er flettet inn i komponentenes API, og det er stikk i strid med det du ønsker.

1. Hva jeg har gjort, og hva som ikke er dekket

Jeg har lest dokumentasjonen (README, CONTRIBUTING, `Klar`-modulen), eksportlisten, API-signaturene til de sentrale modulene (`Table`, `Select`, `Tabs`, `Form`, `Validate`, `Rule`, `Shortcut`, `Date`, `Remote`, `Layout`, `Slider`, `NumberInput`, `TextInput`) og kjørt prosjektets egen API-sjekk. Jeg har **ikke** lest alle 72 000 linjer, ikke kjørt testsuiten på nytt i dag (den var grønn 20. oktober: 4683 tester) og ikke prøvd bevisst feilbruk i en kjørende app. Det som er utledet fra koden, men ikke kjørt, er merket «utledet».

2. Det som er bra

> - **Dype moduler.** `Table.view [ columns, rows, searchable, selectable ]` gir sortering, filtre, paginering, CSV-eksport, tastaturnavigasjon, ARIA og debounce uten at brukeren ser det. Det samme gjelder `Select` (aksent- og store/små-bokstav-løst søk, lange lister), `Format` (korrekt avrunding) og `Search`.
> - **Ensartet API.** Alle komponenter bruker samme mønster: liste med attributter, ugjennomsiktige (opaque) `State` / `Msg`, `update` / `view`. Dette er lett å lære og lett å lese.
> - **Typesikkerhet på de viktigste stedene.** `Locale`, `Icon` og alle attributt-typer er opaque. `Tone`, `Size` og `Density` er lukkede unioner, så «stringly» fargevalg finnes ikke. Ikoner er verdier, ikke navn.
> - **Forsvarlig ved ugyldige tall.** Mange innstillinger forsvares stille: `Slider.step` går gjennom `safeStep`, `Table.server` bruker `max 0`, `Layout.gap` klemmes til skalaen. Ingenting kræsjer (og Elm har ingen unntak).
> - **Ingen HTTP i biblioteket.** Det finnes ingen avhengighet av `elm/http` eller `elm/url`, ingen porter og ingen JavaScript. Bare de fem offisielle pakkene.
> - **Streng prosess.** Egne sjekker for API-flaten, dokumentasjon av alle eksponerte navn, CSS-klasser, ikoner, formatering og kompatibilitet med Elm 0.19.1.
> - **Ærlig dokumentasjon av begrensninger** (kun Chromium testet, ingen mørk modus, ingen skjermlesertest).

3. Funn, i prioritert rekkefølge

#	Funn	Prioritet
1	`Remote` (loading/feil) ligger inne i 15 komponentmoduler	Før 1.0.0
2	Tekst-ID-er overalt: 33 komponenter krever en DOM-id som «må være unik»	Før 1.0.0
3	`Date` er en åpen record, så ugyldige datoer kan lages	Før 1.0.0
4	Åpne `type alias`-records og 21 åpne unioner låser fremtidig API	Før 1.0.0
5	Andre strenger som konfigurasjon: `Shortcut`, kolonne-id, faner, skjemafelt, regelfelt	Bør
6	Oppsett og avlesning av tilstand krever disiplin hos brukeren	Bør
7	Praktisk før publisering (git-repo, testdekning på nettlesere, mørk modus)	Nevnt

Funn 1 – Loading-tilstand hører hjemme utenfor biblioteket (strider mot ønsket ditt)

Du sa at alt som har med http-kall og loading state skal avgjøres utenfor. Slik er det ikke helt i dag. Det er bra at HTTP-kallene selv er utenfor (ingen `elm/http`), men **tilstandsmodellen** er innenfor:

- `Klar.Remote` (`NotAsked | Loading | Failure Problem | Success a`) er en eksponert modul, og 15 komponenter importerer den: `Table`, `Select`, `MultiSelect`, `Autocomplete`, `Tree`, `Timeline`, `Stat`, `Chart`, `ListItem`, `CommandPalette`, `SearchInput`, `InlineEdit`, `Form`, `ConfirmDialog` og `ErrorState`. (`Form`, `ConfirmDialog`, `InlineEdit` og `ErrorState` bruker bare `Problem`, ikke selve tilstanden.)
- API-et er bygd rundt den: `Table.rows : Remote (List row) -> …`, `Select.remote`, `onRetry` på de fleste. Appen din *må* modellere dataene sine som `Remote` for å bruke tabellen, og må gjette hva `NotAsked` skal vises som (biblioteket viser skjelett).
- `Problem` er en åpen record med **ferdig oversatt tekst** (`Remote.problem locale kind` skriver teksten når problemet lages). Skifter brukeren språk, står gammel tekst igjen. Dokumentasjonen innrømmer det selv.
- Dokumentasjonen viser `Http.Error -> Remote.Problem`, så biblioteket legger føringer på hvordan appen strukturerer nettverkslaget.

Anbefaling: komponentene tar vanlige data (`Table.rows : List row`) og ingen `Remote`. Tilstanden «laster» og «feilet» uttrykkes enten som rene visuelle valg (`Table.loading : Bool` på samme måte som `Button.loading`, og et `Table.whenFailed : Html msg`-spor), eller ved at appen velger hva som tegnes (`Skeleton`, `ErrorState` og `EmptyState` er allerede frittstående og kan bli stående). `Klar.Remote` kan fjernes helt, eller flyttes til `example/` som en anbefalt mønsterfil. Avveining: du mister at tabellen tegner et skjelett som passer kolonnene automatisk. Det er den verdien du bør beholde som en ren visuell bryter, uten å eie datamodellen.

Funn 2 – Mange «unike strenger» som kompilatoren ikke kan sjekke

33 stateful komponenter har `init : String -> State`, hvor strengen er en DOM-id som **må være unik på siden** (nevnt 53 ganger i dokumentasjonen). Etiketter, feilmeldinger og fokus bygges på den. Elm kan ikke håndheve unikhet, så to `Select.init "country"` på samme side gir feil kobling mellom etikett og felt, og ingen advarsel. Det samme gjelder `TextInput` uten eksplisitt id: den lager id fra etiketten (`klar-input-<label>`), så to felt med etiketten «Navn» på samme side får samme id (*utledet fra* `ensureId`*, ikke kjørt*).

Anbefaling: (a) gi en testhjelper eller en dokumentert test-oppskrift som finner doble id-er i et `Html`-tre (kan skrives med `elm-explorations/test` i appen); (b) la komponentene utlede id fra posisjon i stedet for å kreve den der den ikke er nødvendig; (c) la id-en være en egen opaque type (`Klar.Id`), slik at man ikke kan blande den med en vanlig tekst som `Tabs`-nøkkelen.

Funn 3 – `Date` er en åpen record

`type alias Date = { year, month, day }` betyr at `{ year = 2025, month = 13, day = 40 }` kompilerer. Det finnes en `Date.isValid` og en trygg `Date.fromParts : … -> Maybe Date`, men ingenting hindrer ugyldige verdier. Typen brukes i `Table.date`, `DatePicker`, `DateRangePicker`, `Calendar` og `Rule`. Dette er det tydeligste «umulige tilstander kan representeres»-tilfellet jeg fant. **Anbefaling:** gjør `Date` opaque før 1.0.0 (aksessorer `year`, `month`, `day` og `fromParts` er allerede der). Etter publisering er det en brytende endring.

Funn 4 – Åpne records og unioner låser API-et

I Elm er det en **hovedversjon** å legge til et felt i en eksponert `type alias`-record eller en konstruktør i en eksponert union. Biblioteket har mange slike aliaser (98 `type alias` i kildekoden, hvorav en del er interne og antallet eksponerte ikke er talt; blant annet `Remote.Problem`, `Shortcut`, `Select.Option`, `Date`, `Table.Query`) og **21 eksponerte unioner** (kommandoen `npm run check:api` lister dem). De mest risikable:

Type	Problem
`Rule.Operator` (25 konstruktører)	Hver ny operator er en hovedversjon. Dette er et åpent sett.
`Remote.Kind` (9), `Rule.FieldKind` (5), `Rule.Operands` (4)	Åpne sett som sannsynligvis vokser.
`Form.Status`, `Table.Filter`, `Table.Selection`, `Calendar.Event`, `Avatar.Status`	Sannsynligvis stabile, men vurder hver for seg.
`Problem`, `Shortcut`, `Option` (records)	Gjør dem opaque med smarte konstruktører, så nye felt ikke er brytende.

Prosjektets egen regel (CONTRIBUTING, punkt 8) sier allerede at åpne sett skal være opaque, men listen over viser at regelen ikke er fulgt overalt. **Anbefaling:** gå gjennom listen én gang og avgjør bevisst hver type. Lukkede, små sett (`Tone`, `Size`) kan bli stående.

Funn 5 – Strenger som konfigurasjon

Du spurte spesifikt om det går an å konfigurere med «bare noen strenger». De viktigste stedene:

Sted	Hva kompilatoren ikke fanger
`Shortcut.shortcut "mod+k" …`	Tastekombinasjonen er fri tekst («mod+k», «g o», «esc»). En skrivefeil som «ctrl+kk» gir en snarvei som aldri utløses. Anbefaling: bygger-funksjoner eller en `Keys`-parser som returnerer `Maybe`.
`Table.text "customer" …` / `setSort [ ("customr", Asc) ]`	Kolonne-id er tekst og gjenbrukes i sortering, filtre, skjulte kolonner og `Query`. En skrivefeil i `setSort` kan ikke fanges på kompileringstid. Anbefaling: la `Table.text` returnere en verdi som kan sendes til `setSort`.
`Tabs.setActive "oversikt"`	Fane-nøkkel er tekst. Dokumentasjonen nevner at en ukjent nøkkel faller tilbake til første fane (bra), men feilen skjules.
`Form.errorSummary { field : String }`, `focusField`	Kobler feilmelding til felt via en id-tekst, så kobling og felt kan komme ut av takt.
`Rule.textField "customer" …`, `Select.Option.option "de" …`	Feltid og verdi er tekst. For verdier er det naturlig (de kommer fra data), men kolonne- og feltid kunne vært typede.
`Layout.gap : Int`	Tallet er egentlig et valg på en skala. Verdier utenfor skalaen klemmes stille. Et lite sett med navngitte verdier ville fjernet muligheten for å skrive feil.

Det betyr **ikke** at appen din kan krasje: Elm har ingen unntak, og biblioteket forsvarer mot de fleste ugyldige verdier. Problemet er at feil ofte blir *stille* i stedet for å være umulige.

Funn 6 – Sammenkobling av state

Hver stateful komponent krever et felt i modellen, en `Msg`-variant, en `update`-gren med `Cmd.map`, og et kall til `view`. Med 33 slike komponenter blir det mye repeterende kode i en stor app. Det er idiomatisk Elm og det er vanskelig å gjøre mye bedre, så jeg vil ikke kalle det en feil. Verre er at **verdien leses ved å sammenligne før og etter** (`Select.value new /= Select.value old`); glemmer man sammenligningen, skjer ingenting og ingen får beskjed. Det er et bevisst valg (ingen out-meldinger), men et typisk sted for stille feil. *Vurder:* en liten hjelper som returnerer «endret?» sammen med ny state, for de komponentene appen oftest reagerer på.

Funn 7 – Praktisk før publisering

- **Ingen git-repo ennå.** `elm publish` krever et offentlig repo med tag `1.0.0`. Stegene står i `RELEASING.md`; jeg har ikke gjort noe av det.
- **Stor flate å stå inne for.** 80 moduler med én vedlikeholder. `Table` alene er 5 700 linjer og `Chart` 3 000. Det er et vedlikeholdsrisiko, ikke et brukerproblem.
- **Globalt reset.** `Klar.stylesheet` forutsetter at biblioteket eier siden. Det er dokumentert, men vil overraske brukere som vil bruke enkeltkomponenter i en eksisterende app.
- **Kjente hull (dokumentert):** kun Chromium testet, ingen mørk modus, ingen skjermlesertest. Det hindrer ikke publisering, men vil komme som issues.

4. Anbefalt rekkefølge

#	Tiltak	Når
1	Fjern `Remote` fra komponent-API-ene (visuelle brytere i stedet). Avgjør om `Klar.Remote` skal forsvinne eller bli et eksempel.	Før 1.0.0
2	Gjør `Date` opaque.	Før 1.0.0
3	Gå gjennom listen fra `npm run check:api` og de eksponerte alias-recordene. Gjør åpne sett (`Rule.Operator`, `Remote.Kind`, `FieldKind`, `Problem`, `Shortcut`) opaque.	Før 1.0.0
4	Typede snarveier, typede kolonne-handles i `Table`, og en sjekk/oppskrift for doble DOM-id-er.	Før 1.0.0 (API), resten kan komme i en minor
5	Kjør `npm run check` og `npm run check:compat` på nytt, opprett git-repo og følg `RELEASING.md`.	Til slutt, bare når du sier fra

Jeg har ikke endret noe i prosjektet i denne gjennomgangen, og jeg har ikke kjørt noe som publiserer eller tagger. Hvis du vil, kan neste steg være å planlegge endringene i punkt 1–3 som en egen jobb, med en oppdatert API-liste og testene som må flyttes.